INTERRUPT INTERACTION WITH FREE RTOS TASK

Assignment 03

CSE323 Advanced Embedded Systems

Table of Contents

1.0 Executive Objective 2
2.0 System Architecture & Task Hierarchy 2
3.0 Hardware Configuration & Implementation 2
3.1 GPIO Initialization 2
3.2 ISR Execution Logic 2
4.0 Debugging & Verification Analysis 3
5.0 Conclusion 5

Figure 1 GPIO interrupt entered before ISR counter update 3
Figure 3 FreeRTOS task woke after ISR notification 4
Figure 2 FreeRTOS task woke after ISR notification 4

1.0 Executive Objective

This report details the integration of hardware interrupts within a Real-Time Operating System (FreeRTOS) environment. The primary objective is to demonstrate the paradigm of deferred interrupt processing. By utilizing a GPIO interrupt on Port F, the system captures a hardware trigger using an inherently minimal Interrupt Service Routine (ISR) and successfully offloads the computational response to a dedicated FreeRTOS task.

2.0 System Architecture & Task Hierarchy

The system architecture was designed to decouple the interrupt detection from the application-level response. The selected hardware trigger is the GPIO Port F interrupt, specifically driven by the onboard switch SW1 on pin PF4.

The software framework is divided into three primary components:

1. **The Hardware Trigger (PF4):** Configured to detect external physical inputs.
2. **The ISR (GPIOF_Handler()):** Designed for minimal execution time. It intercepts the hardware signal and flags the operating system.
3. **The RTOS Tasks:**
    - **vActionTask():** The primary response task that remains in a blocked state, waiting for synchronization from the ISR.
    - **vHeartbeatTask():** A secondary background task implemented to verify that the FreeRTOS scheduler remains active and unblocked during normal system execution.

3.0 Hardware Configuration & Implementation

3.1 GPIO Initialization

Port F was configured with the following parameters to ensure stable hardware interaction:

- **PF4 (Input):** Configured as a digital input utilizing an internal pull-up resistor. It is programmed to generate an interrupt on a falling edge, corresponding to the physical pressing of SW1.
- **PF1, PF2, and PF3 (Outputs):** Configured as standard digital outputs to drive the onboard LED indicators.

3.2 ISR Execution Logic

To maintain RTOS responsiveness, the GPIOF_Handler() performs only the strictly necessary operations before yielding control. Upon a falling edge trigger, the ISR executes the following sequence:

1. Clears the pending interrupt flag for PF4 to prevent recursive ISR loops.
2. Increments the g_interruptCount tracking variable.
3. Toggles the blue LED indicator to provide immediate physical verification of the interrupt firing.

4.0 Debugging & Verification Analysis

A primary engineering challenge during this assignment was empirically verifying the precise synchronization and execution sequence between the hardware ISR and the FreeRTOS task.

To validate this hierarchy, the Keil IDE's debugging environment was heavily utilized. By establishing strategic breakpoints and actively monitoring watch variables (specifically g_interruptCount), it was definitively proven that the ISR successfully preempts normal execution, processes the hardware event, and immediately triggers the scheduler to wake and resume vActionTask().

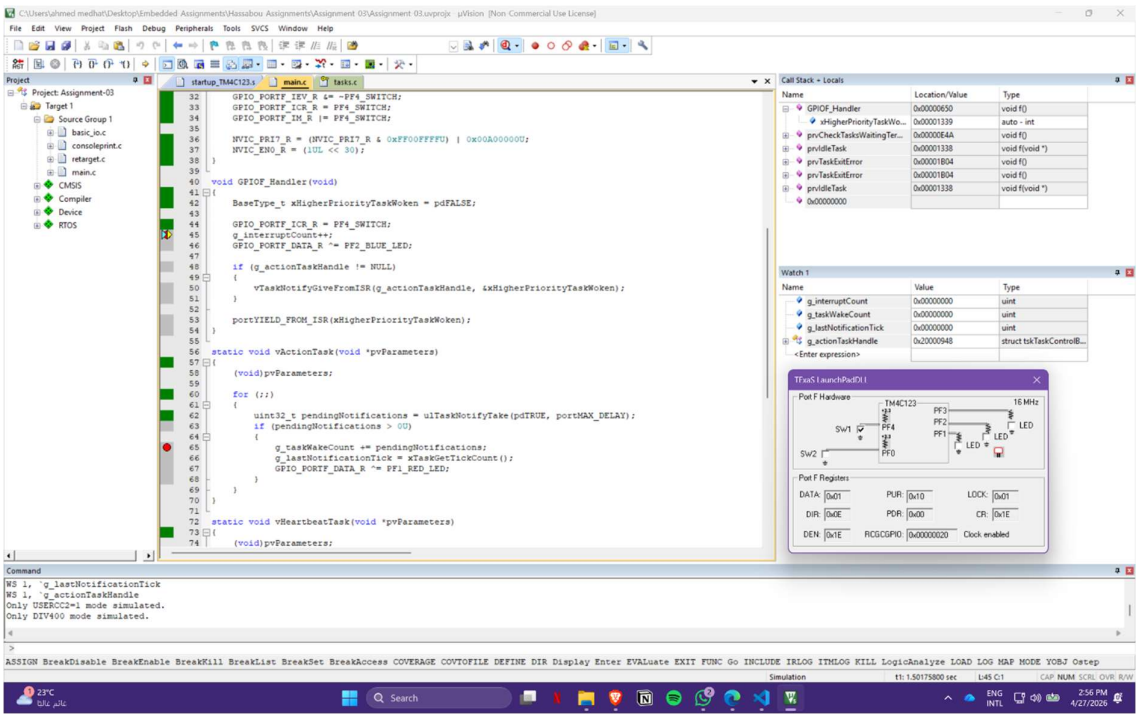

Figure 1 GPIO interrupt entered before ISR counter update

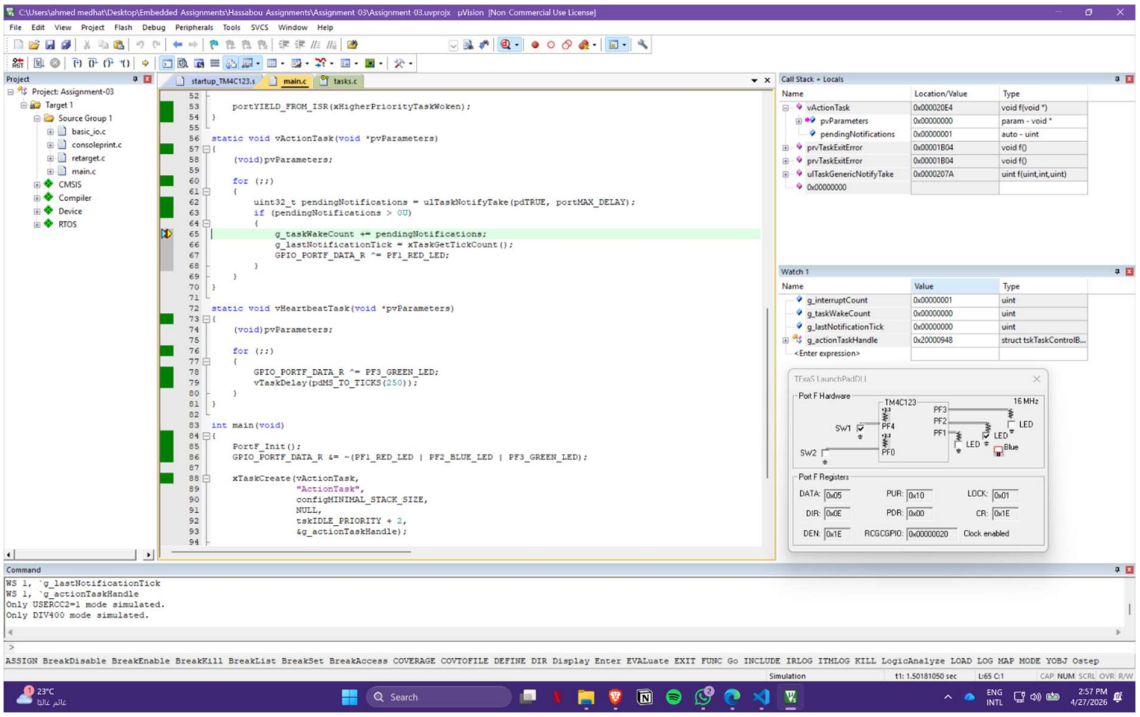

Figure 3 FreeRTOS task woke after ISR notification

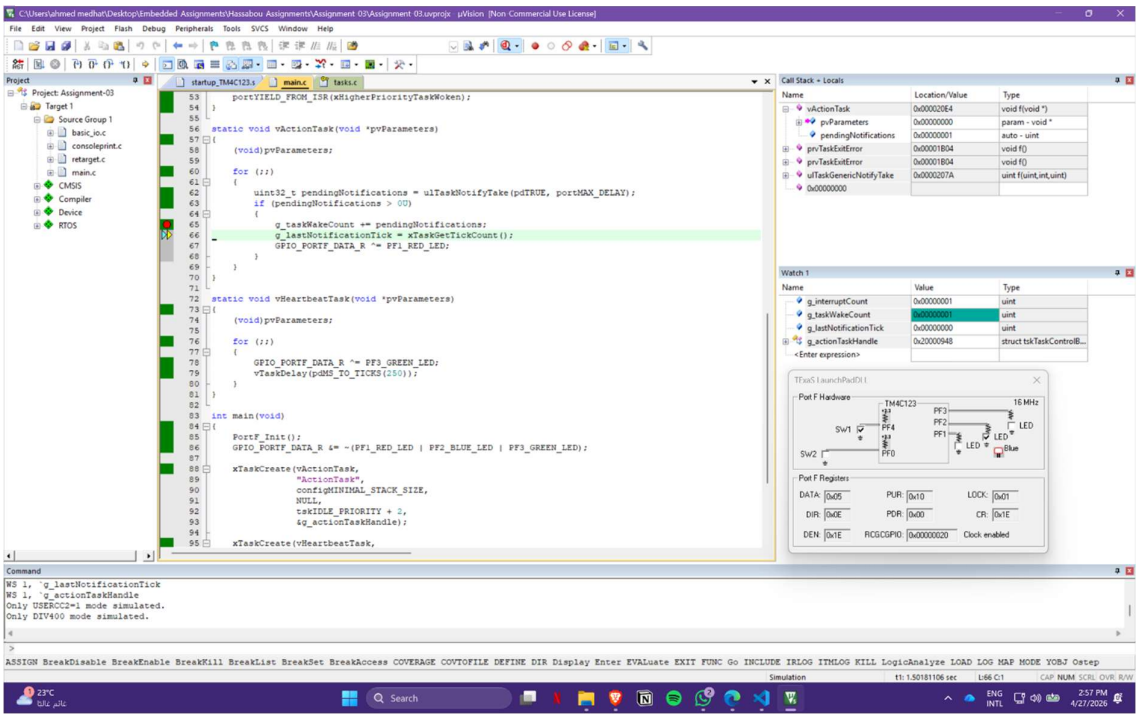

Figure 2 FreeRTOS task woke after ISR notification

5.0 Conclusion

The assignment objectives were successfully achieved. A GPIO interrupt on PF4 was successfully configured and linked to the FreeRTOS ecosystem. The implementation proves that a real-time system can effectively service asynchronous hardware events while preserving scheduler stability and task hierarchy.